

Proyecto Final – Proceso Completo de Modelación

Nombre: Moises Vazquez Sanchez

Nacionalidad: Cubana

Prosedencia: Instituto de Fisica de la UNAM (IFUNAM)

En este trabajo se estudiaron debidamente los ejemplos definidos en el repositorio en github: [lgarciaag89/diplomado_cm_ia](https://github.com/lgarciaag89/diplomado_cm_ia) . Se utilizaron los siguientes conjuntos:

- Entrenamiento: [00-train.csv](#)
- Ajuste: [01-tuning.csv](#)
- Prueba externa: [02-external.csv](#)

1. Selecccion de carateristicas

Para esta primera fase de selección, se implementó el cálculo de **Información Mutua (MI)** [1]. Se eligió esta métrica estadística porque, a diferencia de la correlación de Pearson tradicional, no se limita a buscar relaciones lineales. La Información Mutua es capaz de capturar dependencias complejas y no lineales, midiendo de forma precisa cuánta información pura comparte cada variable independiente. Este análisis nos da una medida matemática de cuánta incertidumbre eliminamos sobre una variable si conocemos el valor de otra.

Realizamos primeramente un mapeo de la base de datos que se utiliza para el entrenamiento y encontramos que la misma posee:

- Variable de objeto: Class
- Cantidad de características iniciales: 95
- Cantidad de registros de entrenamiento: 198

En la informacion suplementaria mostramos los codigo en Python utilizados.

En la Fig. 1 se muestran el Top 10 de las características más informativas según su score de MI.

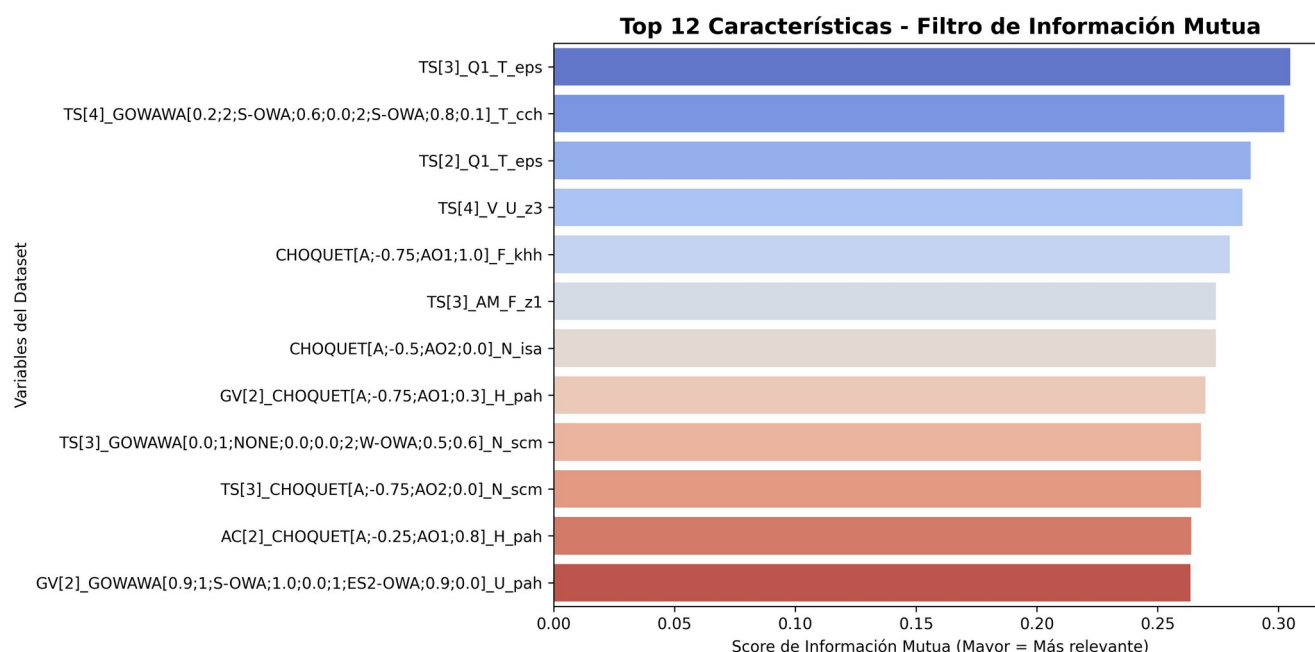


Fig 1. Tabla de Información Mutua de las principales características.

Como podemos ver después del análisis de la Información Mutua la característica más informativa y que tiene un poder de predicción individual mucho más fuerte y directo que las demás es la variable TS[3]_Q1_T_eps (con un score de 0.3049).

Discusión de Estabilidad:

Es importante destacar que este método de filtro es altamente estable. Dado que evalúa cada variable de forma independiente contra la variable objetivo, ligeras variaciones o perturbaciones en el dataset de entrenamiento alterarán muy poco los scores de Información Mutua. Esto contrasta con los métodos tipo Wrapper, que suelen ser más inestables al depender del comportamiento combinatorio y la arquitectura interna de un modelo predictivo.

Análisis de Redundancia por Correlación (Filtro)

Mientras que la MI busca similitud con el objetivo, el **Análisis de Correlación de Pearson** [2] busca relaciones lineales entre las

propias características. El objetivo es detectar qué variables se parecen demasiado entre sí para evitar la redundancia.

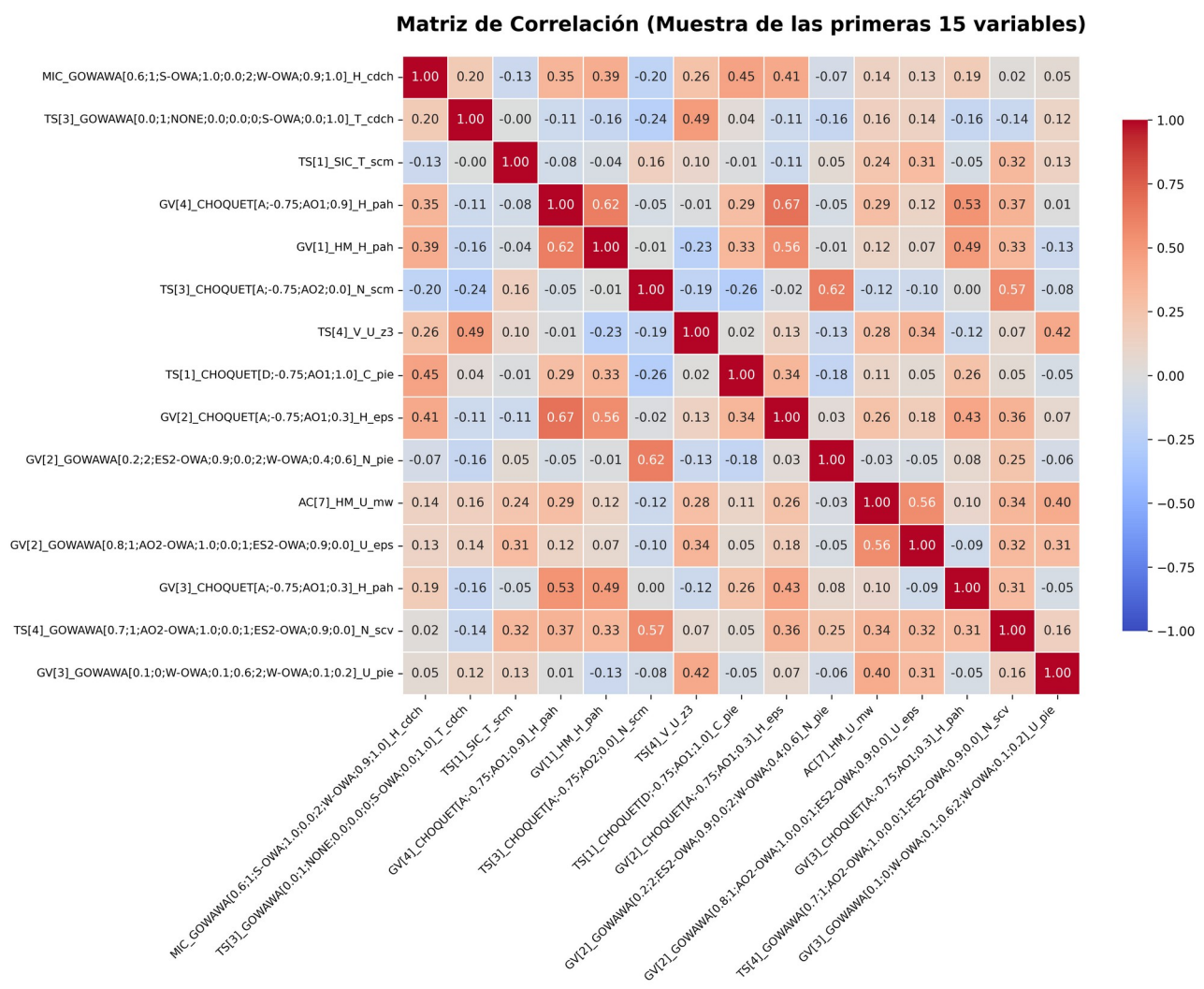


Fig. 2 Matriz de correlacion para el grupo de 77 características.

En la Fig. 2, el cual es un mapa de calor, las celdas con valores cercanos a 1.0 indican que dos variables aportan prácticamente la misma información. Mantener ambas en el modelo genera problemas de multicolinealidad, lo cual confunde severamente a algoritmos matemáticos como la Regresión Logística o SVM.

Para solucionar esto, se aplicó un umbral estricto de eliminación del **0.85**. Se conservó la primera variable de cada par correlacionado y descartó la segunda, garantizando un subconjunto de características eficientes y verdaderamente independientes.

Resultados del Filtro de Correlación:

- **Variables altamente redundantes encontradas ($r > 0.85$):** 18 variables descartadas.
- **Dimensiones finales tras el filtro:** El dataset se redujo de 95 a 77 características útiles.

En la Fig. 2 solo se muestran las primeras 15 variables del conjunto limpio de 77 variables para simplificar la imagen.

Del analisis realizado con este filtro se encontro que 4 de las 10 carateristicas obtenidas con el metodo de MI eran redundantes y por tanto el metodo las descarta. De las 95 caracteristicas totales de la base de datos solo quedaron 77, es decir que se encontraron 18 carateristicas redundantes.

Selección Avanzada mediante Método Wrapper (SFS)

Por su parte el Método Wrapper [3] es una estrategia de selección que no evalúa las variables de forma estadística y aislada, sino que utiliza un algoritmo de Machine Learning real como "juez". Introduce un grupo de variables al modelo, lo entrena, evalúa su capacidad predictiva y repite el proceso iterativamente probando diferentes combinaciones hasta encontrar el equipo óptimo de variables. Utilice la técnica de *Sequential Forward Selection (SFS)* evaluando 10 características finales.

Resultados y Comparación de Subconjuntos (Correlation Subset Merit – CFS)

Se ejecutó el método Wrapper utilizando cuatro estimadores base distintos. Posteriormente, se evaluó la calidad integral de cada subconjunto de 10 variables elegido utilizando la métrica CFS, los resultados obtenidos fueron los siguientes:

- **LogisticRegression:** 0.4441
- **SVM:** 0.5988
- **kNN:** 0.6383
- **RandomForest:** 0.6805

La ecuación del método CFS es la siguiente:

$$Merit_S = \frac{k \cdot \overline{r_{cf}}}{\sqrt{k + k(k - 1) \cdot \overline{r_{ff}}}}$$

Los términos del numerador son: el número de características seleccionadas en ese subconjunto (en nuestro caso 10) y el promedio de la correlación entre las Características y la Forma a predecir (la Clase o variable objetivo), respectivamente. El término del denominador desconocido es el promedio de la intercorrelación entre cada par de características dentro del subconjunto

Discusión y Conclusión de Características:

El subconjunto seleccionado por el algoritmo RandomForest obtuvo el mérito más alto (0.6805), aunque el método KNN obtuvo también buena calificación..

Esta métrica CFS valida matemáticamente que este grupo particular de 10 variables logra el balance perfecto: maximiza la información predictiva hacia la variable 'Class' (numerador de la ecuación CFS) al mismo tiempo que minimiza la redundancia interna entre las características (denominador de la ecuación), superando por un amplio margen a los subconjuntos propuestos por los modelos lineales y espaciales.

2. Modelado (Shallow Learning)

Tras la fase de selección y depuración de características —donde se consolidó un subconjunto óptimo de 10 variables predictoras mediante el método *Wrapper* basado en *Random Forest*— se procedió a establecer la línea base de modelado.

El objetivo de esta sección es evaluar y comparar el comportamiento (con hiperparámetros por defecto) de cuatro algoritmos de aprendizaje supervisado clásico (*Shallow Learning*). Esta evaluación a ciegas

permite identificar qué estructura matemática se adapta mejor a la geometría de los datos antes de realizar cualquier proceso de optimización.

Los cuatro modelos base fueron entrenados exclusivamente con el conjunto de entrenamiento (*Train*) y evaluados de forma justa con el conjunto de validación (*Tuning*). En la Tabla 1 se muestran los resultados:

Modelo Base	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.8548	0.8667	0.8387	0.8525	0.9412
k-NN	0.8387	0.8182	0.8710	0.8438	0.8746
SVM (Lineal)	0.7903	0.7500	0.8710	0.8060	0.9043
Regresión Logística	0.7581	0.7353	0.8065	0.7692	0.9022

Tabla. 1 Comparacion de modelos base (Métricas sobre Tuning Set).

Métricas de Evaluación Utilizadas y Justificación

Para garantizar un análisis robusto y evitar los sesgos que presentan las evaluaciones unidimensionales, el rendimiento de los modelos sobre el conjunto de ajuste (*Tuning Set*) se auditó bajo cinco métricas fundamentales:

- **Accuracy (Exactitud):** Mide la proporción global de aciertos del modelo. Aunque es un indicador intuitivo, no se utiliza de forma aislada para prevenir diagnósticos erróneos en caso de desbalances en la variable objetivo.
- **Precision (Precisión):** Cuantifica la fiabilidad de las predicciones positivas. Es una métrica de control crítica para escenarios donde el costo operativo o el impacto de un **Falso Positivo** sea elevado.

- **Recall (Sensibilidad):** Mide la capacidad del algoritmo para detectar la totalidad de los casos positivos reales. Su optimización es prioritaria para minimizar los **Falsos Negativos**.
- **F1-Score:** Representa la media armónica balanceada entre *Precision* y *Recall*. Es nuestra métrica modelo.
- **ROC-AUC:** Evalúa la capacidad de discriminación estadística de los modelos a través de distintos umbrales de decisión. Un valor cercano a 1.0 indica una separación de clases casi perfecta.

Discusión:

El análisis comparativo de la Tabla.1 revela comportamientos diferenciados que justifican la naturaleza no lineal de nuestro set de datos. El algoritmo de Random Forest demostró el desempeño más sólido y balanceado del proyecto, liderando en cuatro de las cinco métricas evaluadas. Alcanzó una exactitud del **85.48%** y el F1-Score más alto (**0.8525**). Su extraordinario ROC-AUC de **0.9412** confirma que la estructura de ensamble de árboles es altamente eficiente para trazar fronteras de decisión complejas utilizando las 10 variables seleccionadas. Por su parte el modelo KNN se consolidó como el segundo mejor modelo, logrando un F1-Score de **0.8438**. Es destacable su nivel de Recall (**0.8710**), empatando con SVM en la capacidad de identificar la mayor cantidad de positivos reales, aunque su capacidad de discriminación global (ROC-AUC) quedó por debajo de los modelos basados en vectores y árboles. Los modelos lineales (SVM y Regresión Logística) mostraron el rendimiento más modesto. La Regresión Logística, al ser el modelo más desfavorable estadísticamente, se ubicó al fondo de la tabla con un F1-Score de 0.7692. Por su parte, la Máquina de Vectores de Soporte (SVM) con kernel lineal compensó su baja precisión (0.7500) con una excelente sensibilidad (0.8710) y un robusto ROC-AUC de 0.9043, sugiriendo que una separación lineal no es del todo despreciable, pero se ve superada por enfoques no lineales.

3. Optimización de Hiperparámetros

Una vez establecida la línea base del modelado y habiendo identificado el potencial predictivo de los algoritmos clásicos, se procedió a la fase de Optimización de Hiperparámetros. El propósito fundamental de esta etapa es realizar un ajuste fino sobre las configuraciones estructurales de los modelos candidatos para maximizar su rendimiento y capacidad de generalización sobre datos que los modelos nunca han visto, es decir, diferentes a los de entrenamiento.

Para la exploración del espacio de soluciones se contrastaron teóricamente los dos enfoques más extendidos en la literatura científica: Grid Search (Búsqueda en Rejilla) [4] y Random Search (Búsqueda Aleatoria) [5].

- **Grid Search:** Realiza un barrido exhaustivo evaluando de forma determinista la totalidad de las combinaciones posibles dentro de una cuadrícula predefinida. Si bien asegura encontrar el óptimo absoluto dentro de los límites fijados, presenta el inconveniente de un costo computacional exponencial, volviéndose ineficiente al incrementar el número de hiperparámetros.
- **Random Search:** Selecciona configuraciones de forma aleatoria a partir de distribuciones estadísticas sobre un número fijo de iteraciones. Este enfoque estocástico logra mapear las dimensiones más importantes del espacio de búsqueda de forma mucho más eficiente que Grid Search, logrando rendimientos equivalentes en una fracción diminuta de tiempo computacional.

Se seleccionó **Random Search** debido a la alta dimensionalidad de los hiperparámetros configurados (especialmente para *Random Forest*, que involucraba evaluar simultáneamente criterios de división, profundidades y mínimos de muestras). Para este paso del proyecto utilizamos los cuatro modelos que hemos estado utilizando en este proyecto.

Tras la convergencia de la optimización aleatoria, el proceso arrojó las siguientes configuraciones ganadoras para los dos modelos de evaluación que analizamos:

□

Modelo	Hiperparámetros Óptimos Seleccionados	F1-Score (Base)	F1-Score (Optimizado)	Mejora (Δ)
Random Forest	criterion: 'entropy', max_depth: None, min_samples_leaf: 2, min_samples_split: 2, n_estimators: 100	0.8525	0.8710	+0.0185
k-NN	n_neighbors: 5, p: 1, weights: 'distance'	0.8438	0.8571	+0.0133
SVM	C: 100, gamma: 0.01, kernel: 'rbf'	0.8060	0.8182	+0.0122
Regresión Logística	C: 0.1, penalty: 'l1', solver: 'saga'	0.7692	0.8125	+0.0433

Tabla. 2 Impacto de la optimización mediante *Random Search* en el rendimiento predictivo de los modelos candidatos.

Como podemos apreciar en la Tabla. 2, el proceso de búsqueda demostró ser un paso crítico, logrando elevar el umbral predictivo de todos los modelos analizados. Aunque el modelo lineal redujo su brecha de rendimiento gracias a la regularización, los algoritmos de naturaleza no lineal (**Random Forest y k-NN**) demostraron una superioridad innegable para mapear la complejidad de las 10 características seleccionadas. Con estos hiperparámetros definitivos congelados, los cuatro modelos se encuentran en su máxima capacidad, listos para ser evaluados en un análisis de diversidad.

4. Analisis de diversidad

Con el objetivo de evaluar la viabilidad de construir un modelo de ensamble, se calculo las métricas de diversidad entre todas las combinaciones posibles de los algoritmos base previamente optimizados. El cálculo se ejecutó sobre el conjunto de ajuste (*Tuning Set*) para aislar el comportamiento de los modelos frente a datos no vistos durante el entrenamiento.

Para determinar si una combinación es favorable, se evaluaron dos métricas matemáticas complementarias:

1. Métrica de Desacuerdo (Disagreement Measure): Cuantifica la proporción de casos en los que un modelo acierta y el otro se equivoca. Se busca que este valor sea alto, lo que indica que los modelos capturan patrones distintos y aportan información complementaria.
2. Métrica de Doble Fallo (Double Fault Measure): Cuantifica la proporción de observaciones donde ambos modelos se equivocan simultáneamente. Se busca que este valor sea lo más bajo posible para asegurar que el ensamble no replique los mismos errores.

Los cálculos arrojaron los siguientes resultados para las 6 combinaciones posibles:

Modelo A	Modelo B	Desacuerdo (↑)	Doble Fallo (↓)
Regresión Logística	SVM	0.0645	0.1612
Regresión Logística	k-NN	0.1451	0.0967
Regresión Logística	Random Forest	0.0967	0.1129
SVM	k-NN	0.0806	0.1290

Modelo A	Modelo B	Desacuerdo (↑)	Doble Fallo (↓)
SVM	Random Forest	0.1290	0.0967
k-NN	Random Forest	0.0483	0.1129

Tabla.3 Métricas de desacuerdo y doble fallo para combinaciones de modelos evaluadas en el *Tuning Set*.

Basados en los valores obtenidos, se concluye que sí es altamente recomendable y matemáticamente justificable crear un ensamble para mejorar el rendimiento global del sistema. Específicamente, combinar modelos de naturaleza distinta que obtuvieron alto desacuerdo y bajo doble fallo (como el par de **SVM y Random Forest** o un ensamble heterogéneo que incluya a **Regresión Logística y k-NN**) permitirá construir un clasificador más robusto. Esta mezcla compensará las debilidades individuales de los algoritmos y superará muy probablemente el límite predictivo del mejor modelo individual.

5. Construcción de modelos por ensamble

Con base en los resultados favorables obtenidos en la matriz de diversidad, se procedió a la construcción de arquitecturas predictivas complejas mediante técnicas de aprendizaje en conjunto. El objetivo principal de esta fase fue determinar si la combinación de algoritmos lograría superar el umbral de rendimiento establecido por el mejor modelo individual (Random Forest optimizado, con un F1-Score de 0.8710).

Para este último paso del proyecto final se construyeron e instrumentaron tres modelos de ensamble distintos:

1. **AdaBoost [6]:** Algoritmo de impulso secuencial clásico.
2. **XGBoost [7]:** Algoritmo avanzado de *Gradient Boosting*.

3. Ensamble Manual (Votación Suave / Soft Voting): Un comité heterogéneo construido exclusivamente con los tres algoritmos base más fuertes de la fase anterior (Random Forest, k-NN y SVM), promediando sus probabilidades de clase.

Los modelos fueron entrenados y posteriormente evaluados sobre el conjunto de ajuste (*Tuning Set*), arrojando las siguientes métricas de desempeño:

Modelo Ensemble	Accuracy	Precision	Recall	F1-Score	ROC-AUC
AdaBoost	0.9194	0.9062	0.9355	0.9206	0.9292
Ensamble Manual (Votación Soft)	0.8871	0.9000	0.8710	0.8852	0.9157
XGBoost	0.8387	0.8621	0.8065	0.8333	0.9126

Tabla. 4 Desempeño global (Accuracy, Precision, Recall, F1-Score y ROC-AUC) de los métodos *AdaBoost*, *XGBoost* y Ensamble Manual.

Al contrastar el rendimiento de los ensambles contra los modelos base, podemos declarar categoricamente que la creación del ensamble manual mediante votación suave fue un éxito. Al registrar un F1-Score de **0.8852**, logró superar al Random Forest individual (**0.8710**). Esto confirma el análisis de diversidad previo: las debilidades del SVM y las limitaciones geométricas del k-NN fueron efectivamente compensadas por las reglas del Random Forest, creando un sistema conjunto superior a sus partes individuales. Por su parte el algoritmo de Gradient Boosting moderno obtuvo el rendimiento más bajo del bloque (F1-Score de **0.8333**). Esto sugiere que, con su configuración estándar, el modelo pudo haber sobreajustado los datos de entrenamiento o que requiere un ajuste de hiperparámetros mucho más profundo para adaptarse a este conjunto de características en particular. El algoritmo de AdaBoost emergió como el modelo indiscutiblemente superior de todo el proyecto.

Logró un salto cuantitativo extraordinario, elevando el **F1-Score a 0.9206** y el **Accuracy a 0.9194**. Su capacidad para corregir secuencialmente los errores de los iteradores anteriores demostró ser la estrategia más afín a la topología de los datos. Además, su métrica ROC-AUC de **0.9292** garantiza una excelente capacidad para distinguir entre las clases bajo distintos umbrales de decisión.

Conclusion:

A modo de conclusion se confirma categóricamente que los modelos por ensamble contruidos sí mejoran el rendimiento de los modelos base. La transición de algoritmos individuales a arquitecturas conjuntas permitió romper la barrera predictiva del 87%, llevando el sistema a un nivel de precisión y robustez superior al **92%** gracias a la implementación de AdaBoost. En consecuencia, se recomienda la selección de **AdaBoost** como el modelo final y definitivo para su despliegue en este problema de clasificación. Tambien el ensamble manual mostro superioridad predictiva con respecto a los modelos vaces previamente entrenados.

Bibliografía:

- [1] Peng, H., Long, F., & Ding, C. (2005). Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8), 1226-1238.
<https://doi.org/10.1109/TPAMI.2005.159>
- [2] Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). *Probabilidad y estadística para ingeniería y ciencias* (9.^a ed.). Pearson Educación.
- [3] Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1-2), 273-324.
[https://doi.org/10.1016/S0004-3702\(97\)00043-X](https://doi.org/10.1016/S0004-3702(97)00043-X)
- [4] Bergstra, J., & Bengio, Y. (2012). Random search for hyperparameter optimization. *Journal of Machine Learning Research*, 13(2), 281-305.
- [5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. (Nota: El Capítulo 11 de este libro detalla exhaustivamente las diferencias teóricas entre Grid Search y Random Search).
- [6] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119-139.
<https://doi.org/10.1006/jcss.1997.1504>
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).

Association for Computing Machinery.
<https://doi.org/10.1145/2939672.2939785>